



Rapport de laboratoire

# Traitement d'images par réseau

INF3405 - Réseaux informatiques

|                       |                                      |
|-----------------------|--------------------------------------|
| <b>Session</b>        | Été 2026                             |
| <b>Projet</b>         | TP1 - Traitement d'images par réseau |
| <b>Date de remise</b> | 31 mai 2026                          |
| <b>Soumis à</b>       | Ikram Kohil / Fatima-Azahrae Bikrani |

## Équipe

| <b>Matricule</b> | <b>Nom</b>               |
|------------------|--------------------------|
| 2464064          | Sonfack Ngoune Roosevelt |

## Introduction

Ce travail pratique avait pour objectif de développer une application réseau client-serveur en Java permettant d'envoyer une image à un serveur, d'y appliquer un filtre de Sobel, puis de retourner l'image traitée au client. Le projet permettait de mettre en pratique les notions de sockets, de communication binaire, de validation d'entrées, de gestion de fichiers et d'exécution concurrente avec plusieurs clients.

L'application réalisée est composée de deux programmes console : un client et un serveur. Le client se connecte au serveur avec une adresse IP, un port, un nom d'utilisateur et un mot de passe. Le serveur authentifie l'utilisateur ou crée automatiquement son compte si celui-ci n'existe pas, reçoit l'image, applique le filtre de Sobel, puis retourne le résultat au client.

## Présentation de la solution

La solution est développée uniquement en Java, sans bibliothèque externe pour la logique principale. Les échanges réseau sont faits avec **Socket** côté client et **ServerSocket** côté serveur. Les données sont transmises à l'aide de **DataInputStream** et **DataOutputStream**, ce qui permet d'envoyer clairement les chaînes de caractères, la taille de l'image et les octets du fichier.

Au démarrage, le serveur demande l'adresse IP locale et le port d'écoute. Les ports acceptés sont limités à l'intervalle demandé, soit 5000 à 5050. La validation de l'adresse IP vérifie que l'adresse contient exactement quatre octets numériques et que chaque octet est compris entre 0 et 255. La même validation est réutilisée côté client pour éviter d'essayer une connexion avec des paramètres incohérents.

Le serveur gère les utilisateurs avec une base de données locale dans le fichier **users.db**. Lorsqu'un nom d'utilisateur n'existe pas, le serveur crée automatiquement le compte et associe le mot de passe fourni. Lorsqu'un utilisateur existe déjà, le mot de passe fourni doit correspondre à celui enregistré. Cette base est persistante sur disque, donc les comptes restent disponibles après l'arrêt et le redémarrage du serveur.

Pour permettre plusieurs connexions en même temps, le serveur accepte les clients dans une boucle et délègue chaque client à un thread géré par un **ExecutorService**. Chaque client peut donc envoyer une image sans bloquer complètement le serveur. Lorsqu'une image est reçue, le serveur affiche dans la console le nom de l'utilisateur, l'adresse IP et le port du client, la date et l'heure, ainsi que le nom de l'image reçue.

Le traitement d'image est effectué côté serveur avec la classe **Sobel.java**, conformément à l'énoncé. L'image reçue est décodée avec **ImageIO**, traitée par l'algorithme de Sobel, puis réencodée dans un format compatible avec le nom de fichier demandé par le client. Le client enregistre ensuite l'image traitée sur disque et affiche le chemin absolu du fichier reçu.

## Difficultés rencontrées

La première difficulté a été de passer d'une logique de développement web à une communication réseau plus bas niveau. En web, les requêtes HTTP structurent déjà les échanges, alors qu'ici il fallait définir nous-mêmes l'ordre exact des données envoyées entre le client et le serveur. Il fallait donc s'assurer que le client et le serveur lisent les champs dans le même ordre : identifiants, nom du fichier, taille de l'image, contenu binaire, puis réponse du serveur.

Une autre difficulté a été la transmission fiable des images. Contrairement à une chaîne de caractères simple, une image doit être envoyée comme un tableau d'octets. La solution a été d'envoyer d'abord la taille du fichier, puis exactement le nombre d'octets correspondant. Cela permet au serveur et au client de détecter une réception incomplète.

La gestion de plusieurs clients a aussi demandé une attention particulière. Le serveur ne devait pas traiter un seul client à la fois, car une image volumineuse ou une connexion lente pourrait bloquer les autres utilisateurs. L'utilisation d'un pool de threads a permis de séparer chaque connexion tout en gardant un serveur capable de rester en écoute.

Enfin, la validation des entrées console a demandé plus de rigueur que dans une interface web habituelle. Dans une application web, les champs de formulaire peuvent être typés, stylés et validés côté navigateur. Ici, il fallait gérer explicitement les erreurs de saisie dans la console, notamment pour les adresses IP, les ports et les noms de fichiers.

## Critiques et améliorations

Le laboratoire est utile pour comprendre ce qui se passe sous les abstractions utilisées en développement web. Il serait toutefois intéressant d'ajouter une courte comparaison avec HTTP, car cela aiderait les étudiants qui ont déjà une expérience web à relier les sockets Java aux concepts plus familiers comme les requêtes, les réponses, les statuts et les corps de messages.

---

Une amélioration possible serait de fournir un protocole minimal attendu entre le client et le serveur, par exemple l'ordre des champs ou un format de message recommandé. Cela laisserait encore de la place à l'implémentation, mais réduirait les erreurs liées à des lectures et écritures désynchronisées.

Il serait aussi pertinent d'ajouter une option permettant au client d'envoyer plusieurs images dans une même session, au lieu de se déconnecter après un seul traitement. Cela rapprocherait davantage le projet d'un vrai service réseau et permettrait de mieux explorer la gestion de sessions.

## Conclusion

Ce laboratoire m'a permis de mieux comprendre la communication réseau avec les sockets Java et la différence entre une application web déjà structurée par un protocole comme HTTP et une application où le protocole d'échange doit être conçu manuellement. J'ai aussi consolidé ma compréhension de la concurrence côté serveur, de la persistance simple sur disque et du transfert de fichiers binaires.

Le projet a été utile parce qu'il relie plusieurs notions concrètes : validation des paramètres, authentification, stockage local, threads, traitement d'image et échange client-serveur. Il montre aussi l'importance d'un protocole clair entre deux programmes, car une petite différence entre l'ordre d'écriture et l'ordre de lecture peut empêcher toute la communication de fonctionner correctement.